Object Detection App

Project Plan and System Architecture

Project goal: Build a webcam application that detects objects from a defined set of classes and displays each detected object's name, bounding box, and confidence score. The model is trained or fine-tuned using an annotated dataset, evaluated, exported, and integrated into a real-time inference application.

Scope: This is a mid-tier computer-vision project. The app detects only the classes represented by its model. It does not recognize every possible object or identify people. Model choice and performance targets must be validated on the intended computer and camera.

1. Learning and build plan

Project stages

1. Define the task. Select the object classes and intended camera setting; agree on user-visible behavior and how success will be measured. **Output:** project brief, class list, and acceptance criteria.

2. Prepare a dataset. Collect representative images with appropriate permission; annotate boxes and class IDs; review labels and create leakage-aware train, validation, and test splits. **Output:** versioned dataset and annotation rules.

3. Train a YOLO model. Learn the single-stage detection concept, use a suitable pretrained checkpoint, fine-tune on project classes, and record the run configuration. **Output:** reproducible training run and selected checkpoint.

4. Evaluate. Measure precision, recall, per-class AP, mAP@0.5, and mAP@0.5:0.95; inspect errors and prediction examples. **Output:** evaluation summary and error review.

5. Build live inference. Capture webcam frames, preprocess to model input, infer, postprocess boxes, and draw object names and confidence scores. **Output:** working preview with controls and error handling.

6. Export and compare. Export to a format supported by the selected runtime; compare detections and accuracy with the original checkpoint and measure latency on target hardware. **Output:** verified deployment artifact and benchmark notes.

7. Release and own. Document setup, supported environment, model/data provenance, class limits, privacy behavior, licensing checks, and issue handling. **Output:** versioned release and maintenance handoff.

2. Problem definition and requirements

Object detection locates instances of known classes in an image and assigns each box a class and confidence score. The model's class list is a product boundary: objects outside that list may not be detected or may be assigned incorrectly. A generic pretrained model detects its trained categories; adding project-specific classes requires suitable labeled examples and model training or fine-tuning.

Functional requirements

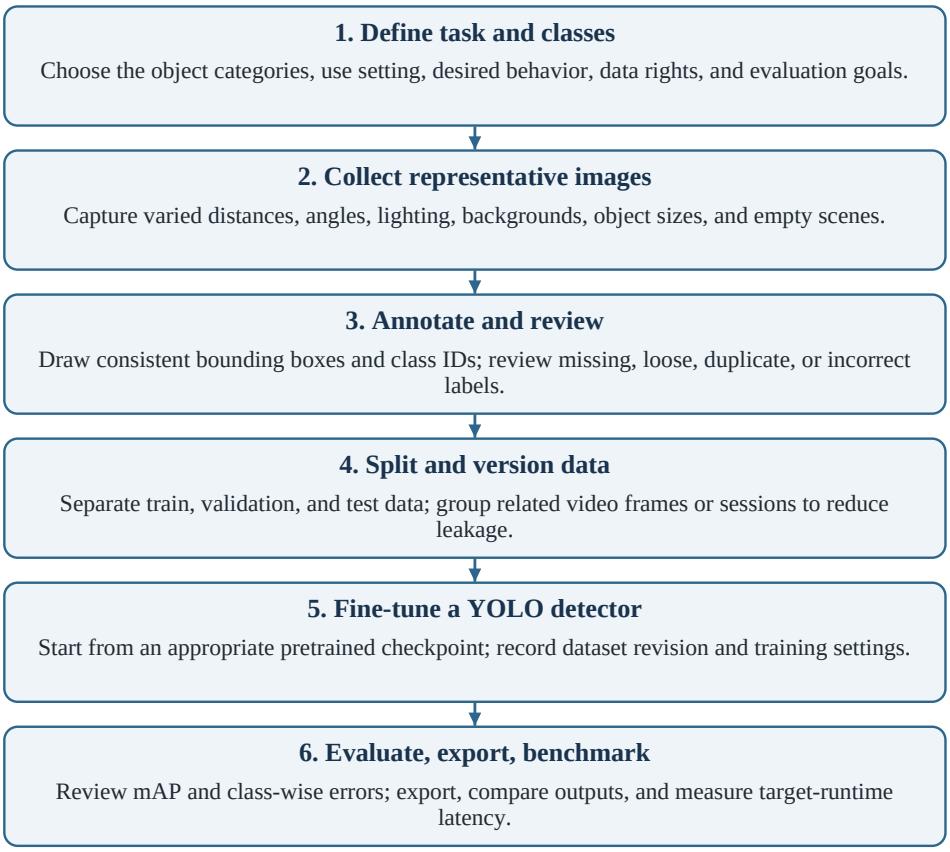
- Open one webcam and show a live preview.
- Run object detection on incoming frames and draw a bounding box, class name, and confidence for each retained detection.
- Allow users to quit and, if included, adjust camera selection or confidence threshold.
- Show a useful error when the camera cannot open, frame reading fails, or the model cannot load.
- Load model weights and class metadata once at startup and release camera/window resources on exit.

Quality and privacy requirements

- Responsiveness: measure capture, preprocessing, inference, postprocessing, display, and total loop time on target hardware. Do not promise a frame rate before measuring.
- Reliability: validate frames and detections; prevent malformed outputs from drawing outside the displayed image.
- Privacy: process webcam frames locally; do not record or upload them by default.
- Maintainability: separate camera I/O, preprocessing, model execution, postprocessing, UI, and configuration.
- Reproducibility: record dataset version, class map, model/checkpoint, training settings, dependencies, and exported runtime format.

3. Training data and annotation plan

Define the classes and annotation rules before collecting the main dataset. Collect images that reflect expected use: different object sizes, distances, angles, backgrounds, lighting, partial occlusion, and empty scenes. Avoid treating near-identical frames from one video as independent evidence across train and test splits.



If test results are weak, inspect errors and improve data or training; do not tune against the final test split.

Figure 1. Dataset, training, evaluation, and export lifecycle.

Annotation and data quality checks

- Use one bounding box and class ID per object. In YOLO detection text labels, represent each object with class ID and normalized center-x, center-y, width, and height; follow the selected framework's current format guide. Decide how to label partial, overlapping, and ambiguous objects.
- Review a sample of labels visually for missing objects, wrong classes, duplicate boxes, loose or clipped boxes, and inconsistent edge cases.
- Keep train, validation, and test images in distinct splits. Group related images from the same recording session/source before splitting to reduce leakage.
- Document image sources, consent or usage rights, annotation tool/output format, class map, and dataset revision.
- Preserve an untouched test set for final evaluation; use validation data for model and threshold decisions.

4. YOLO model and training concepts

YOLO refers to a family of object detectors designed to predict object locations and classes efficiently in a single model inference path. A training workflow typically starts from a pretrained checkpoint and fine-tunes it on the project's labeled classes. The model processes an image tensor and produces candidate boxes and class scores; the selected framework handles model-specific decoding and may apply non-maximum suppression (NMS) to remove duplicate overlapping predictions.

Training decisions students should understand

- Model size/input resolution: trade model capacity and accuracy against compute, memory, and latency. Choose after establishing target hardware and a baseline.
- Transfer learning: reuse learned visual features from pretrained weights, then adapt them to project classes. Record the source checkpoint and its terms.
- Augmentation: vary training images in ways that resemble plausible camera conditions; inspect that transformations preserve meaningful labels.
- Class balance: measure examples and instances per class; investigate poor class-specific performance rather than relying on total image count.
- Overfitting: compare training and validation behavior; improve data quality or coverage when validation does not generalize.

5. Evaluation and mAP

Intersection over Union (IoU) measures the overlap between a predicted box and a ground-truth box. Precision describes the share of predicted detections that are correct; recall describes the share of ground-truth objects that are found. Average Precision (AP) summarizes a class's precision-recall behavior. Mean Average Precision (mAP) averages AP across classes. [3]

Metric	Interpretation for this project
Precision / recall	Use together to understand false detections versus missed objects; review each class separately.
AP per class	Summarizes detection quality for one class across confidence settings, at a specified IoU evaluation rule.
mAP@0.5	Mean AP across classes when a predicted box is considered a match at IoU 0.50. More permissive about box overlap. [3]
mAP@0.5:0.95	Mean AP across classes and IoU thresholds from 0.50 through 0.95, in 0.05 steps; it reflects localization quality at stricter overlaps as well. [3]

Metric	Interpretation for this project
Latency / throughput	Measure separately from accuracy; runtime format and device can change speed. Report test hardware, image size, and timing method.

Do not use one mAP score as the release decision. Review class-wise metrics, precision-recall behavior, confusion/errors, labeled prediction examples, and live webcam behavior. Select the confidence threshold using validation data and the application's acceptable false-alert/missed-object tradeoff; report final results on the held-out test split.

6. Runtime architecture

The runtime path starts at the webcam and ends at an annotated preview. The detector is loaded once. Every frame passes through preprocessing that must match training/export assumptions; results are postprocessed and mapped back to displayed-frame coordinates before drawing.

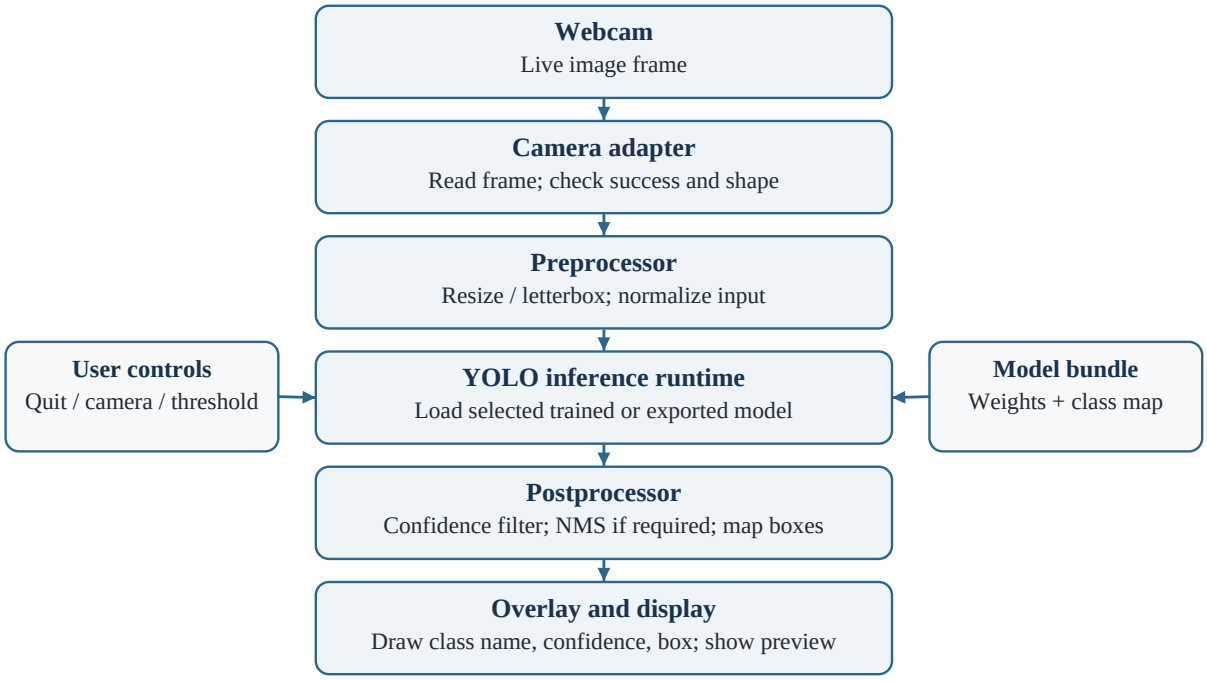


Figure 2. Webcam inference architecture and model/config inputs.

Runtime component contracts

Component	Input → output / responsibility
Camera adapter	Camera device → valid frame or explicit read error. Owns open/read/release.
Preprocessor	Frame → model input tensor, including resize/letterbox, color order, normalization, and size required by the model.
Detector adapter	Preprocessed input → raw or framework-decoded detections. Hides whether the backend is a training framework or exported runtime.
Postprocessor	Candidate outputs → validated detections after confidence filtering, NMS when required, class lookup, and coordinate mapping.
Renderer/controller	Frame plus detections/status → preview; handles controls, errors, loop timing, and cleanup.

7. Frame inference sequence

For each frame: capture → validate → preprocess → infer → postprocess → render → display → poll controls. If inference is slower than camera capture, use an intentional policy such as processing the latest available frame rather than building an unbounded queue; measure the end-to-end effect before choosing.

Postprocessing safeguards

- Apply a documented confidence threshold; do not silently hard-code a threshold that users cannot inspect.
- Apply NMS only once when needed by the chosen model/export. Some export paths can include postprocessing; verify the exported model’s output contract to avoid duplicated NMS.
- Validate box coordinates, class IDs, and score ranges before rendering. Clip box coordinates to image boundaries.
- Map model-input coordinates back through resize/letterbox transforms to original frame coordinates. Test this with known synthetic boxes.
- Use the class map bundled with the exact model version; never assume numeric class IDs have the same meanings across models.

8. Model export and deployment

Keep the original training checkpoint as the source artifact. Export a separate deployment artifact only after selecting a runtime and target hardware. ONNX is one common interchange format; other formats can be preferable for particular devices. The selected YOLO framework documents available export formats and options, and recommends measuring accuracy and speed for the deployment target. [1]

Artifact / metadata	Purpose
Training checkpoint	Supports reproducibility, later fine-tuning, and recovery; retain its provenance and license terms.
Exported model	Runtime-specific weights, for example ONNX where supported; record precision, input shape, opset/runtime requirements as applicable.
Class map	Ordered mapping from model class IDs to display names; version with the model.
Inference configuration	Input size, confidence threshold, IoU/NMS settings, and any class filters used by the app.
Model card / run manifest	Dataset revision, training code/config, checkpoint source, framework/runtime versions, evaluation summary, export method, target device, and known limitations.

Export verification gate

- Run the original checkpoint and exported artifact on the same held-out images and compare class IDs, scores, and box coordinates within documented tolerances.
- Recompute evaluation metrics using the exported runtime where possible; compare against the original rather than assuming export preserves results exactly.
- Benchmark warm and steady-state inference using documented hardware, image size, batch size, and timing boundaries; include preprocessing/postprocessing if reporting application-level latency.
- Test webcam integration and failure behavior with the exported model before release.
- Review the code, pretrained-weight, dataset, and export-runtime licenses/terms before redistribution.

9. Testing, release, and ownership

Test strategy

- Unit tests: class-map lookup, confidence filtering, NMS wrapper behavior, coordinate restoration, and invalid output rejection. Use synthetic arrays and known boxes.
- Data tests: label parser, class IDs in range, valid normalized box geometry, matching image/label pairs, and no accidental source overlap across splits.
- Integration tests: recorded non-sensitive sample images through the selected runtime; webcam smoke tests on supported operating systems.
- System tests: camera unavailable, model file missing/corrupt, window close, quit key, model inference error, and resource cleanup.
- Performance tests: separate capture, preprocess, inference, postprocess, and full-loop timing; record target hardware and input resolution.

Release and ownership checklist

- README with environment setup, model acquisition/export steps, controls, supported platforms, troubleshooting, and privacy behavior.
- Version-controlled source, pinned tested dependencies, model metadata, and documented artifact provenance.
- Versioned release with known limitations and measured evaluation/performance summary.
- Issue reports request version, OS/device, model ID, reproduction steps, and errors; do not request webcam footage by default.
- Assign a maintainer to review dependencies, model/data provenance, compatibility, and user-reported failures.

References

[1] Ultralytics. “Model Export with Ultralytics YOLO.” https://docs.ultralytics.com/modes/export/

[2] Ultralytics. “Object Detection Datasets Overview.” https://docs.ultralytics.com/datasets/detect/

[3] Ultralytics. “YOLO Performance Metrics.” https://docs.ultralytics.com/guides/yolo-performance-metrics/

[4] Ultralytics. “Object Detection with Ultralytics YOLO.” https://docs.ultralytics.com/tasks/detect/

Technical documentation links reviewed October 2026. Confirm current package, model, export-format, and license terms when selecting implementation dependencies.